

ICS 604

Applied Data Science

Department of Information and Computer Sciences
University of Hawai`i at Mānoa

Kyungim Baek

Lecture 4

- Arithmetic Operations and Data Alignment
 - Vectorized arithmetic
 - Subsetting based on logical conditions
- Data Wrangling: Data Preparation
 - Inspecting and modifying data types (column)
 - String methods and method chaining

Vectorization

- A vectorized operation applies to a list of elements at once.
 - Implemented in the NumPy library.
 - Easily extends to Pandas, which uses NumPy to store data.
 - Taking advantage of the parallel capabilities of modern CPUs, it is much more efficient than Python's *for loops*.
 - Automatically handles collection of data in parallel.

Vectorization (cont'd.)

- A Series contains homogeneous data types (though it can technically hold mixed data types).
 - Python typically works with lists of heterogeneous datatypes.
- Series operations are delegated to optimized and compiled C code.
 - This process is known as *vectorization*. The result is a tremendous speedup compared to an analogous computation in Python, which:
 - Iterates through the arrays
 - Each item is painstakingly checked for its data type

```
x = pd.Series(range(1_000_000))
display(x.head())
x.shape
```

```
0    0
1    1
2    2
3    3
4    4
dtype: int64
(1000000,)
```

```
%%time
y = x**2
```

```
CPU times: user 5.66 ms, sys: 6.99 ms, total: 12.7 ms
Wall time: 7.72 ms
```

```
%%time
a = range(1_000_000)
b = []
for elem in a:
    b.append(elem**2)
```

```
CPU times: user 80.6 ms, sys: 10.8 ms, total: 91.5 ms
Wall time: 88 ms
```

The numerical Python (NumPy) library

- NumPy is a Python library for array-based computing implemented in the C language.
 - The *de facto* standard for scientific computing.
 - Foundation for the Python scientific stack.
- Pandas uses numpy to store the data behind the scenes.

```
x = pd.Series([1, 2, 3])
print(type(x.values))

<class 'numpy.ndarray'>
```

*S. to-numpy()
df. "*

- Numpy objects have various methods to work with the data they contain.

Broadcasting

- Broadcasting is a set of rules for applying binary functions (e.g., addition, subtraction, multiplication, etc.) on arrays of different shapes.
- Broadcasting follows a strict set of rules to determine the interaction between the two arrays.
 - When working with arrays in one dimension, the rules are fairly simple:
 1. If the shape of the two arrays does not match, the array with a shape equal to 1 is stretched to match the second array.
 2. If sizes disagree and neither is equal to 1, an error is raised.

```
x = np.array([1, 2, 3])
y = np.array([3, 2, 1])

print(x.shape, y.shape)
x + y
```

```
(3,) (3,)
array([4, 4, 4])
```

```
x = np.array([1, 2, 3])
y = np.array([1, 2])
print(x.shape, y.shape)
x + y
```

```
(3,) (1,)
array([2, 3, 4])
```

Second condition throws and error

```
x = np.array([1, 2, 3])
y = np.array([1, 2])
```

```
print(x.shape, y.shape)
x + y
```

```
(3,) (2,)
```

```
ValueError                                Traceback (most recent call last)
Cell In[10], line 6
      3 y = np.array([1, 2])
      5 print(x.shape, y.shape)
----> 6 x + y

ValueError: operands could not be broadcast together with shapes (3,) (2,)
```

Broadcasting rules

1D 2D

Suppose we have numpy arrays **x** and **y**. How will they broadcast?

- Write down the shape of each array as a tuple of integers.
 - For example, **x**: (10,), **y**: (20, 10)
- If they have different numbers of dimensions, *prepend* with 1s until they have the same number of dimensions.
 - For example, **x**: (10,) → **x**: (1, 10) **y**: (20, 10)
- Compare each dimension. There are 3 cases: $\neq$
 - a) Dimension match. Everything is good.
 - b) Dimensions don't match, but one is = 1.
"Duplicate" the smaller array along that axis to match.
 - a) Dimensions don't match, neither are = 1. Error!

Slide credit: Justin Johnson & David Fouhey

Broadcasting: Examples

```
x = np.ones((10, 20))
y = np.ones(20)
z = x + y
print(z.shape)
(10, 20)
```

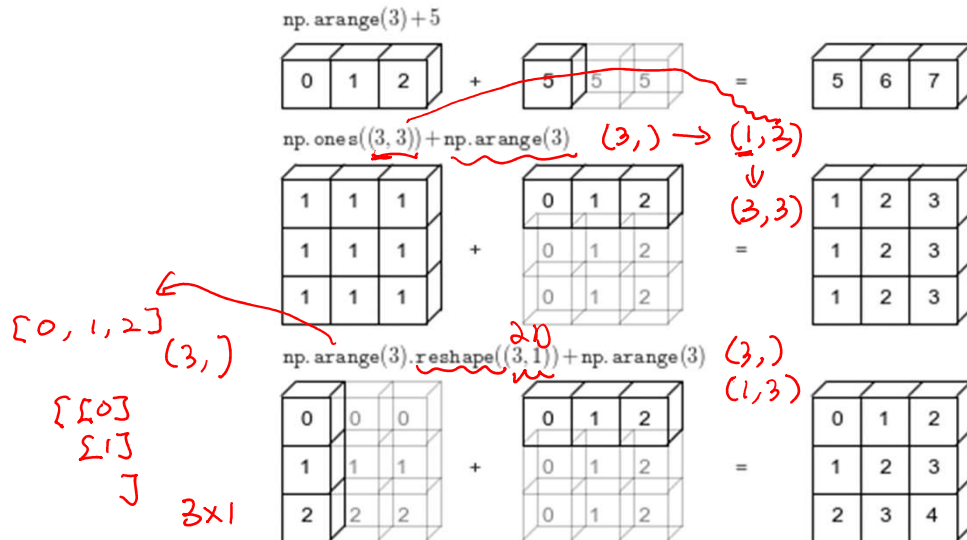
```
x = np.ones((10, 20))
y = np.ones((10, 1))
z = x + y
print(z.shape)
(10, 20)
```

```
x = np.ones((10, 20))
y = np.ones(10)
z = x + y
print(z.shape)
ERROR
```

```
x = np.ones((1, 20))
y = np.ones((10, 1))
z = x + y
print(z.shape)
(10, 20)
```

Slide credit: Justin Johnson & David Fouhey

Broadcasting: Examples (cont'd.)



Using `numpy.newaxis` for broadcasting

- `np.newaxis` is a special object in numpy used to add a new axis (or dimension) to an existing array, effectively changing its shape.
- What will be the output of the following code?

```
import numpy as np

arr = np.array([1, 2, 3])
result = arr[:, np.newaxis] + np.array([0, 10, 20])
print(result)
```

(t) $\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 3 & 3 & 3 \end{bmatrix}$ $\begin{bmatrix} 0 & 0 & 20 \\ 0 & 0 & 20 \end{bmatrix}$

Vectorization: Arithmetic operations

- Arithmetic operations in Pandas are vectorized.
 - You can sum or multiply two columns and the operation would be vectorized, i.e., parallelized.
- We explore using a trivial dataset:

```
df_1["AA"] + df_1["BB"]
```

```
A    90  
C     4  
T    15  
X    30  
dtype: int64
```

```
df_1 = pd.DataFrame({'AA': {'A': 79, 'C': 2, 'T': 13, 'X': 21},  
                     'BB': {'A': 11, 'C': 2, 'T': 2, 'X': 9}})  
df_1
```

	AA	BB
A	79	11
C	2	2
T	13	2
X	21	9

Arithmetic operations (cont'd.)

- Vectorized arithmetic is not limited to operations on the data with the same index or Series of the same size

```
df_2 = pd.DataFrame({'AA': {'A': 21, 'D': 14, 'T': 5},  
                     'CC': {'A': 12, 'D': 28, 'T': 121}})  
df_2
```

	AA	CC
A	21	12
D	14	28
T	5	121

```
df_1["AA"] + df_2["AA"]
```

```
A    100.0  
C      NaN  
D      NaN  
T     18.0  
X      NaN  
Name: AA, dtype: float64
```

Vectorized arithmetic between Series (DataFrame columns)

- Column-wise vectorized operations require aligning the data on the index.
 - A new index is created from the *union* of the indices of both Series.
 - Missing values in either one of the Series are filled with missing values (NaN).

df_1 =	AA		BB				
	A	79	11	df_1["AA"] + df_2["AA"]			
	C	2	2				
	T	13	2				
	X	21	9				
df_2 =	AA		CC	=			
	A	21	12				
	D	14	28				
	T	5	121				

A	79	21	A	100
C	2	NaN	C	NaN
D	NaN	14	D	NaN
T	13	5	T	18
X	21	NaN	X	NaN

Vectorized arithmetic between Series (DataFrame rows)

- Operations on row Series work the same way as columns.

df_1 =	AA		BB				
	A	79	11	df_1.loc["A"]			
	C	2	2				
	T	13	2				
	X	21	9				
df_2 =	AA		CC	df_2.loc["D"]			
	A	21	12				
	D	14	28				
	T	5	121				

AA	79	14	AA	93
BB	11	NaN	BB	NaN
CC	NaN	28	CC	NaN

```
df_1.loc["A"] + df_2.loc["D"]
```

```
AA    93.0
BB     NaN
CC     NaN
dtype: float64
```


Vectorized arithmetic on DataFrames

- We can also perform arithmetic operations on the whole DataFrame.

df_1 =

	AA	BB
A	79	11
C	2	2
T	12	2
X	21	9

df_2 =

	AA	BB
A	21	12
D	14	28
T	5	121

	AA	BB
A	79	11
C	2	2
D	NAN	NAN
T	12	2
X	21	9

+

	AA	BB
A	21	12
C	NAN	NAN
D	14	28
T	5	121
X	NAN	NAN

=

	AA	BB
A	100	23
C	NAN	NAN
D	NAN	NAN
T	17	123
X	NAN	NAN

df_1 + df_2 =

```
df_1 = pd.DataFrame({'AA':{'A':79, 'C':2, 'T':12, 'X':21},
                     'BB':{'A':11, 'C':2, 'T':2, 'X':9}})
df_2 = pd.DataFrame({'AA':{'A':21, 'D':14, 'T':5},
                     'BB':{'A':12, 'D':28, 'T':121}})
df_1 + df_2
```

	AA	BB
A	100.0	23.0
C	NaN	NaN
D	NaN	NaN
T	17.0	123.0
X	NaN	NaN

Question

- What would the outcome be if a column is present in one DataFrame and not in the other?
- Example:

df_1

	AA	BB
A	79	11
C	2	2
T	12	2
X	21	9

+

df_2

	AA	CC
A	21	12
D	14	28
T	5	121

=?

Question (cont'd.)

- What would the outcome be if a column is present in one DataFrame and not in the other?

data frame and not in the other.

df_1

	AA	BB
A	79	11
C	2	2
T	12	2
X	21	9

+

df_2

	AA	CC
A	21	12
D	14	28
T	5	121

=

df_1 + df_2

	AA	BB	CC
A	100.0	NaN	NaN
C	NaN	NaN	NaN
D	NaN	NaN	NaN
T	17.0	NaN	NaN
X	NaN	NaN	NaN

Vectorization example with the medical spending dataset

- An arithmetic operation between two Series yields a Series.
 - We can use the resulting Series in new computations.
- Example: We can compute the average spending per beneficiary of the `spending_df` DataFrame using the following steps:
 1. For each `unique_id` (row), we can divide spending by `nb_beneficiaries`.
 - This gives us the spending for each beneficiary for each `unique_id`.
 2. Compute the mean of the resulting values.

```
spending_df = pd.read_csv('data/spending_10k.csv', index_col='unique_id')
spending_df.head()
```

	doctor_id	specialty	medication	nb_beneficiaries	spending
unique_id					
NX531425	1255626040	FAMILY PRACTICE	METFORMIN HCL	30	135.24
QG879256	1699761833	FAMILY PRACTICE	ALLOPURINOL	30	715.76
FW363228	1538148804	INTERNAL MEDICINE	LOSARTAN POTASSIUM	146	1056.47
WD733417	1730200619	PSYCHIATRY	OLANZAPINE	13	28226.97
XW149832	1023116894	FAMILY PRACTICE	PRAVASTATIN SODIUM	348	8199.48

```
spending_benef = spending_df['spending'] / spending_df['nb_beneficiaries']
type(spending_benef)
```

```
pandas.core.series.Series
```

```
spending_benef
```

```
unique_id
NX531425    4.508000
QG879256    23.858667
FW363228    7.236096
WD733417    2171.305385
XW149832    23.561724
...
ZK471712    7.081061
QM412803    7.266515
NN242894    3.525000
ER452896    51.684865
WH309636    2.113333
Length: 10000, dtype: float64
```

per ben. spending

```
print(spending_benef.mean())
```

```
131.9261641934523
```

```
# operation chaining
```

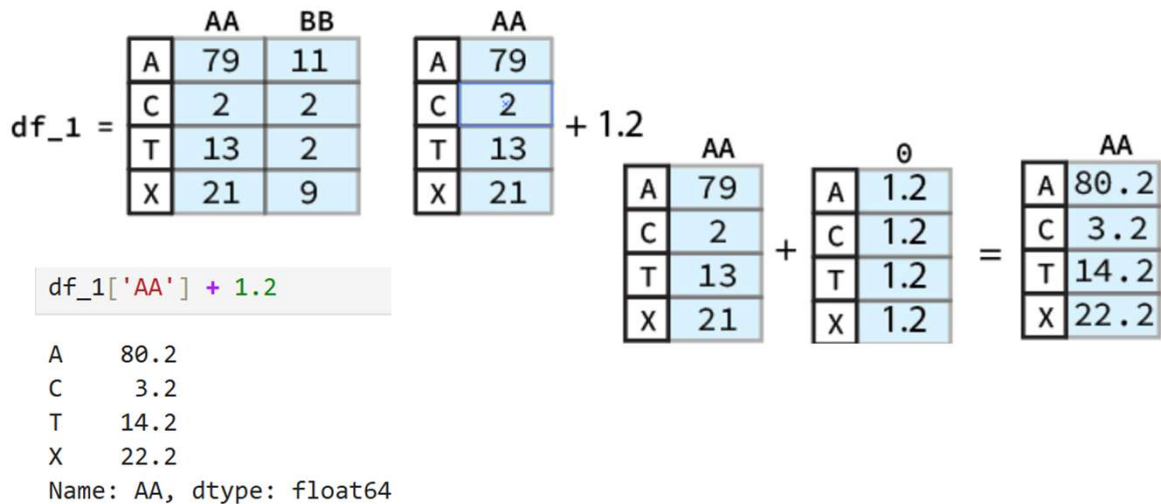
```
print((spending_df['spending'] / spending_df['nb_beneficiaries']).mean())
```

```
131.9261641934523
```

Broadcasting

- What happens with an arithmetic operation that involves a Series and a single value (scalar)?
 - The logic is the same for DataFrame and a single value.
- The scalar is expanded (broadcasted) to fit the dimension of the Series (or DataFrame).
- Example: `df_1['AA'] + 1.2`

Scalar broadcasting



Series-to-DataFrame broadcasting

- First, align the Series index with one of the DataFrame's axes based on matching labels.
- By default, align with DataFrame columns.

```
s = pd.Series([100, 200], index=['AA', 'BB'])
df_1 + s
```

	AA	BB
A	179	211
C	102	202
T	113	202
X	121	209

AA 100
BB 200

	df_1	
	AA	BB
A	79	11
C	2	2
T	13	2
X	21	9

Series-to-DataFrame broadcasting (cont'd.)

```
# Align s with the row labels of df_1 by specifying the axis
s = pd.Series([100, 200], index=['A', 'C'])
df_1.add(s, axis=0)
```

	AA	BB
A	179.0	111.0
C	202.0	202.0
T	NaN	NaN
X	NaN	NaN

```
s = pd.Series([100, 200, 300], index=['A', 'B', 'C'])
df_1.add(s, axis=0)
```

	AA	BB
A	179.0	111.0
B	NaN	NaN
C	302.0	302.0
T	NaN	NaN
X	NaN	NaN

	df_1		
		AA	BB
A	100	79	11
B	200 X		
C	300	2	2
T		13	2
X		21	9

Handwritten notes and arrows illustrating the broadcasting process:

- A red circle around 'A' with '100' next to it.
- A red circle around 'B' with '200 X' next to it.
- A red circle around 'C' with '300' next to it.
- Red arrows pointing from the circled values to the corresponding rows in the 'df_1' table.

Conditional subsetting

- We often need to subset a dataset and we know how to subset a dataset to extract a single row.

```
spending_df.iloc[1]
```

- We know how to subset a dataset to extract a range of rows.

```
spending_df.iloc[1:32]
```

- But it's also very useful to subset the data based on some condition:
 - Extract all the entries in cardiology.
 - How many flights out of HNL were going to Nebraska?
 - How many Biki bicycles were checked out in a specific geographical region (e.g., Manoa)?

Comparison operations

- Comparison operators (<, >, ==, >=, <=, !=) are applied in the same fashion as arithmetic operations.
 - Result in Booleans (True or False)

```
df_1 = pd.DataFrame({'AA':{'A':79, 'C':2, 'T':13, 'X':21},  
                    'BB':{'A':11, 'C':2, 'T':2, 'X':9}})  
df_3 = pd.DataFrame({'AA':{'A':64, 'C':2, 'T':6, 'X':22},  
                    'BB':{'A':3, 'C':2, 'T':7, 'X':12}})
```

```
df_1["AA"] > df_3["AA"]
```

```
A      True  
C     False  
T      True  
X     False  
Name: AA, dtype: bool
```

Comparison result

- Result of the comparison is a Series with the same indices in df_1 and df_3.
 - **IMPORTANT:** Comparisons between two Series or DataFrames can only be carried out if **both objects are identically labeled**.
 - Have the same row **and** column names.
- All entries are of type bool.
 - True is the position passed the comparison test and False otherwise.

Comparison result (cont'd.)

```
## Not the same number of rows nor the same index
```

```
df_4 = pd.DataFrame({'AA':{'I':64, 'J':2, 'K':6}})
```

```
df_1["AA"] > df_4["AA"]
```

ValueError Traceback (most recent call last)

Cell In[31], line 5

```
1 ## Not the same number of rows nor the same index
3 df_4 = pd.DataFrame({'AA':{'I':64, 'J':2, 'K':6}})
----> 5 df_1["AA"] > df_4["AA"]
```

⋮

ValueError: Can only compare identically-labeled Series objects

Comparison operations and indexing

- Comparison operators are ideal for querying and subsetting.
 - SQL-like functionality although at much lower efficiency (both RAM and CPU intensive compared to dedicated DBMS).
- We can subset a Series using a Boolean list ([]) that has the same size.

```
x = pd.Series([1, 2, 3])
```

```
my_filter = pd.Series([True, False, True])
```

```
x[my_filter]
```

```
0    1
2    3
dtype: int64
```

index Value		
0	1	T
1	2	F
2	3	T

Comparison operations and indexing (cont'd.)

```
df_1 = pd.DataFrame({'AA':{'A':79, 'C':2, 'T':13, 'X':21},
                    'BB':{'A':11, 'C':2, 'T':2, 'X':9}})
```

```
boolean_list = [True, True, False, False]
df_1.loc[boolean_list, 'AA']
```

```
A    79
C     2
Name: AA, dtype: int64
```

	df_1	
	AA	BB
A	79	11
C	2	2
T	13	2
X	21	9

Comparison operations and indexing (cont'd.)

- Row *(Selection)* filtering

df_1 =

	AA	BB
A	79	11
C	2	2
T	13	2
X	21	9

True
False
True
False

	AA	BB
A	79	11
T	13	2

```
df_1[[True, False, True, False]]
```

	AA	BB
A	79	11
T	13	2

```
df_1['AA'] > 3
```

```
A    True
C   False
T    True
X    True
Name: AA, dtype: bool
```

```
df_1[df_1['AA'] > 3]
```

	AA	BB
A	79	11
T	13	2
X	21	9

Boolean operations

- Conditional expressions allow us to create more sophisticated queries for our dataset.
 - Queries commonly rely on Boolean logic (Boolean operators).
- Pandas' Boolean operations are slightly different from Python's.
 - and is replaced with `&`
 - or is replaced with `|`
 - not is replaced with `~`
- The operators above return a Boolean object of the same shape as the input.

Boolean operations: Example

- Find instances in `df_1` where values in columns "AA" and "BB" are greater than 3.

```
true_false_series = (df_1['AA'] > 3) & (df_1['BB'] > 3)
true_false_series
```

```
A    True
C    False
T    False
X    True
dtype: bool
```

	AA	BB
A	79	11
C	2	2
T	13	2
X	21	9

```
df_1[true_false_series]
```

	AA	BB
A	79	11
X	21	9

```
df_1[(df_1['AA'] > 3) & (df_1['BB'] > 3)]
```

	AA	BB
A	79	11
X	21	9

Boolean operations examples with the medical spending dataset

- Let's subset the `spending_df` to retain rows that:
 - Have 'DENTIST' as specialty
 - Have spending below \$100
- Save results to a new DataFrame.

Example cont'd.

- Have 'DENTIST' as specialty:

`spending_df['specialty'] == 'DENTIST'`

`(spending_df['specialty'] == 'DENTIST').head()`

```
unique_id
NX531425    False
QG879256    False
FW363228    False
WD733417    False
XW149832    False
Name: specialty, dtype: bool
```

Example cont'd.

- Have spending below \$100:

```
spending_df['spending'] < 100
```

```
(spending_df['spending'] < 100).head()
```

```
unique_id  
NX531425    False  
QG879256    False  
FW363228    False  
WD733417    False  
XW149832    False  
Name: spending, dtype: bool
```

Example cont'd.

- Combine both conditions:

```
(spending_df['specialty'] == 'DENTIST') & (spending_df['spending'] < 100)
```

```
((spending_df['specialty'] == 'DENTIST') & (spending_df['spending'] < 100)).head()
```

```
unique_id  
NX531425    False  
QG879256    False  
FW363228    False  
WD733417    False  
XW149832    False  
dtype: bool
```

Example cont'd.

- Subset the original dataset:

```
dentist_and_small_spending = (spending_df['specialty'] == 'DENTIST') & (spending_df['spending'] < 100)
spending_df.loc[dentist_and_small_spending].head()
```

	doctor_id	specialty	medication	nb_beneficiaries	spending
unique_id					
XY759578	1114930567	DENTIST	IBUPROFEN	23	67.19
NR408938	1407936230	DENTIST	IBUPROFEN	20	64.20
KK703203	1669464095	DENTIST	CLINDAMYCIN HCL	14	94.95
RI403710	1063709178	DENTIST	HYDROCODONE/ACETAMINOPHEN	14	74.48
YM646458	1528120383	DENTIST	AMOXICILLIN	23	63.84

Data preparation: About the data

- The dataset `spending_clean_ex.csv` was specifically constructed from the Medicare dataset.
- It was modified to contain interesting cleaning use cases.

```

spending_df = pd.read_csv('data/spending_clean_ex.csv', index_col='unique_id')
spending_df.head(7)

```

	doctor_id	specialty	medication	nb_beneficiaries	spending
unique_id					
BK982218	1750389599	INTERNAL MEDICINE	AZITHROMYCIN	12.0	\$77.26
CG916968	1952344418	CARDIOLOGY	SIMVASTATIN	85.0	\$767.83
SA964720	1669522744	INTERNAL MEDICINE	INSULIN DETEMIR	14.0	\$5,409.29
TR390895	1639597115	NaN	LOSARTAN POTASSIUM	11.0	\$65.62
JA436080	1073781571	NaN	LAMOTRIGINE	12.0	\$8,873.70
WO822855	1689080749	INTERVENTIONAL CARDIOLOGY	SERTRALINE HCL	26.0	NaN
PC409862	1568483196	INTERNAL MEDICINE	CLOTRIMAZOLE/BETAMETHASONE DIP	12.0	\$310.97

Inspecting and modifying data types

- Recall that `dtypes` returns data type of each column.

```
spending_df.dtypes
```

```

doctor_id      int64
specialty      object
medication      object
nb_beneficiaries float64
spending        object
dtype: object

```

- Issues:
 - `doctor_id` is saved as an `int64`.
 - `spending` is saved as an `object`.